

Ingeniería del Software

Tema 1. Panorámica

TABLA DE CONTENIDOS

01

Breve Historia

02

¿Qué es la Ingeniería del Software?

03

Ciclo de vida

04

Herramientas CASE

05

Diagramas

TABLA DE CONTENIDOS

01

Breve Historia

02

¿Qué es la Ingeniería del Software?

03

Ciclo de vida

04

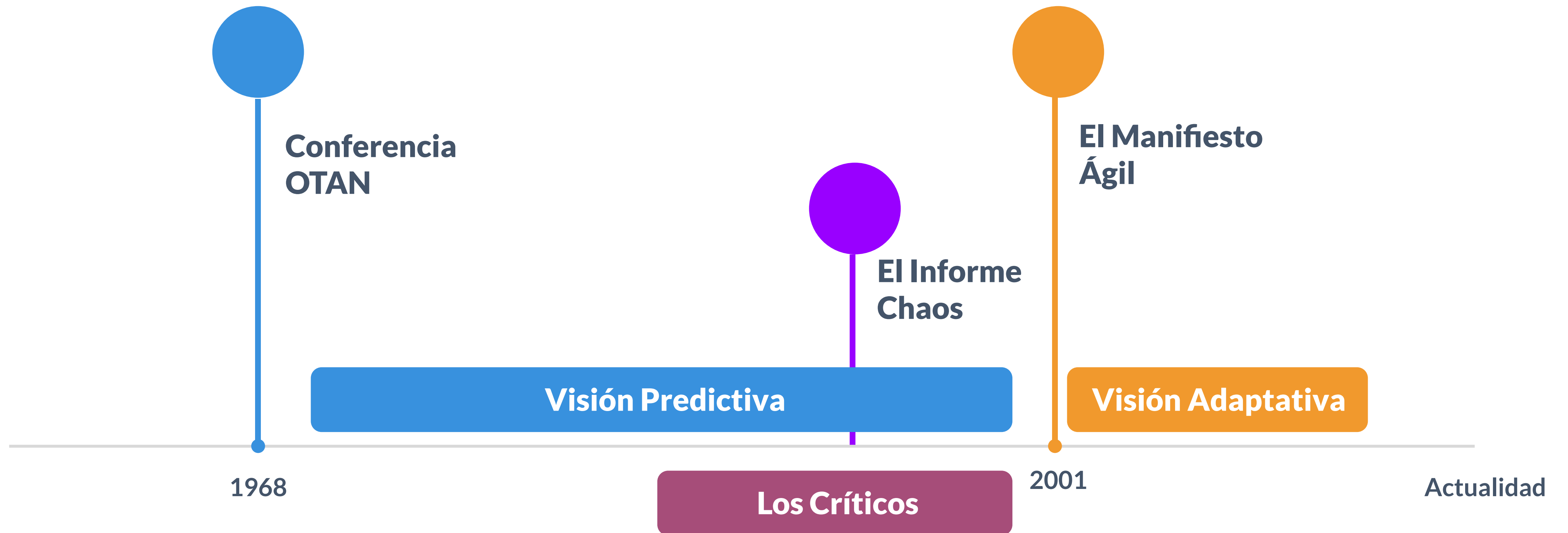
Herramientas CASE

05

Diagramas

Breve Historia

Del Apollo, al Ejército de Simios



Del Apollo, al Ejército de Simios



Margaret Hamilton, **la pionera**

En 1964, Margaret Hamilton fue la jefa de desarrollo del software de la primera misión espacial norteamericana, **Apollo**.

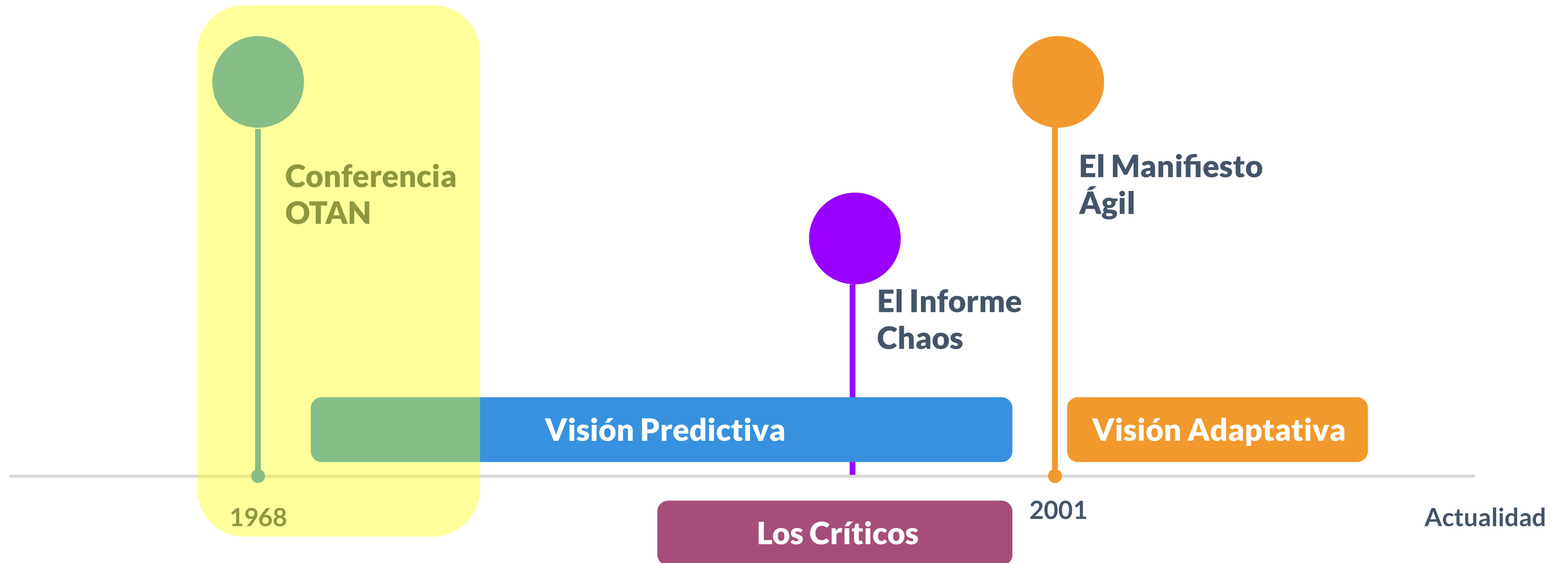
“Comencé a usar el término Ingeniería de Software para distinguirlo del hardware y otros tipos de ingeniería. Trate cada tipo de ingeniería como parte de un proceso de más amplio alcance que unía ingenierías de otros sistemas”

- Margaret Hamilton



Margaret Hamilton con el código del Apollo

Del Apollo, al Ejército de Simios



Conferencia de la OTAN, el nacimiento de la Ingeniería de Software



Conferencia de la OTAN, en Garbisch, Alemania, 1968

En 1968 se identificó al desarrollo del software como una actividad caótica en la construcción de grandes sistemas. Se acuñó el término “**crisis del software**” para definir la situación, y se acordó la necesidad de establecer procesos de ingeniería para el desarrollo de software.

“La Ingeniería de Software es la parte de la ciencia de la computación demasiado difícil para el científico computacional.”

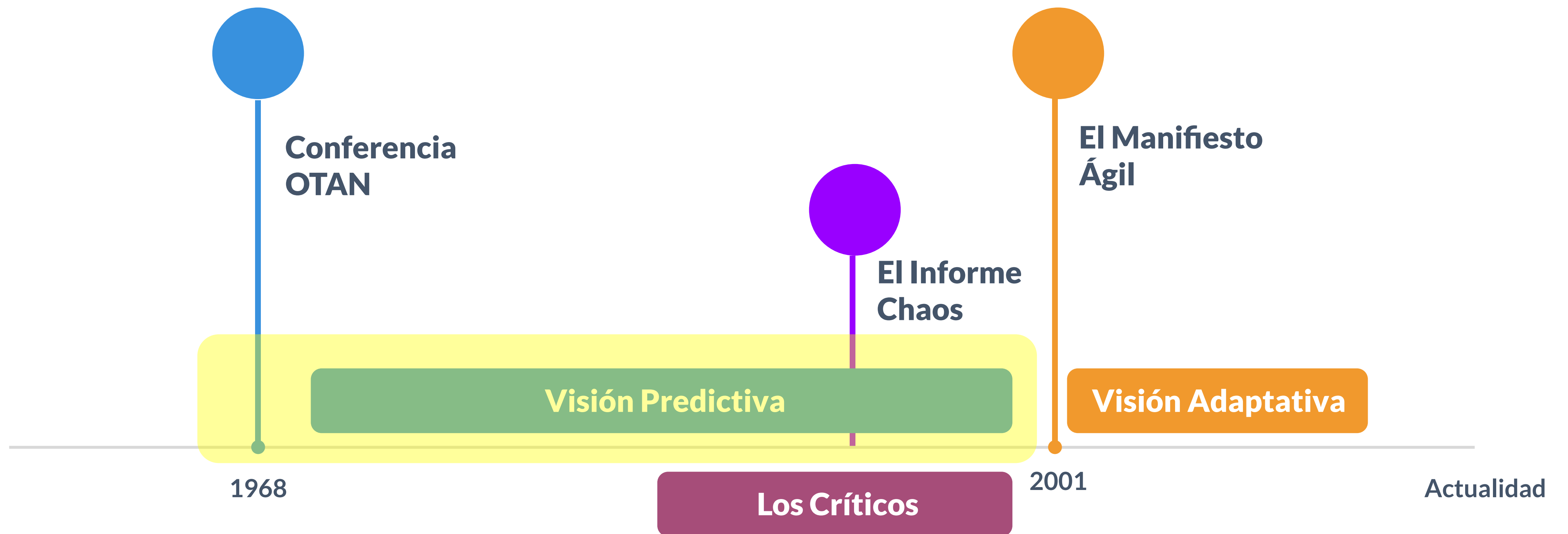
- Friedrich Bauer



La industria manufacturera ha conseguido sistematizar sus metodologías de construcción dotando de previsibilidad al proceso y generando productos de calidad y económicamente rentables, ¿debería la recién definida Ingeniería de Software asimilar dichas metodologías para construir productos software de calidad y baratos? ¿debemos construir software de la misma forma que construimos puentes?

- A) **Sí.** El producto software debe construirse igual que se construyen coches, casas o puentes.
- B) **No.** El producto software debe seguir construyéndose de forma artesanal, casi artística

Del Apollo, al Ejército de Simios

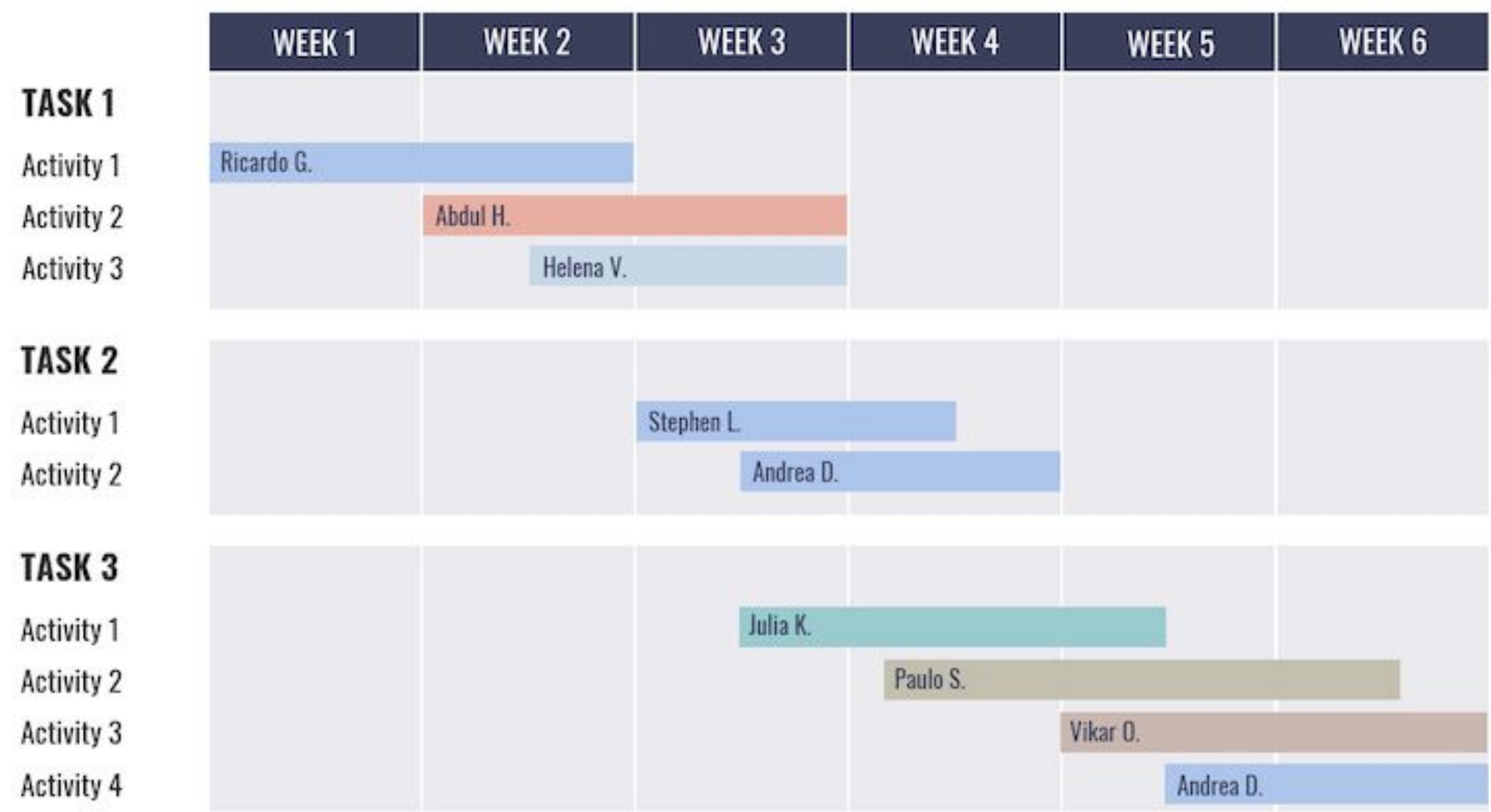




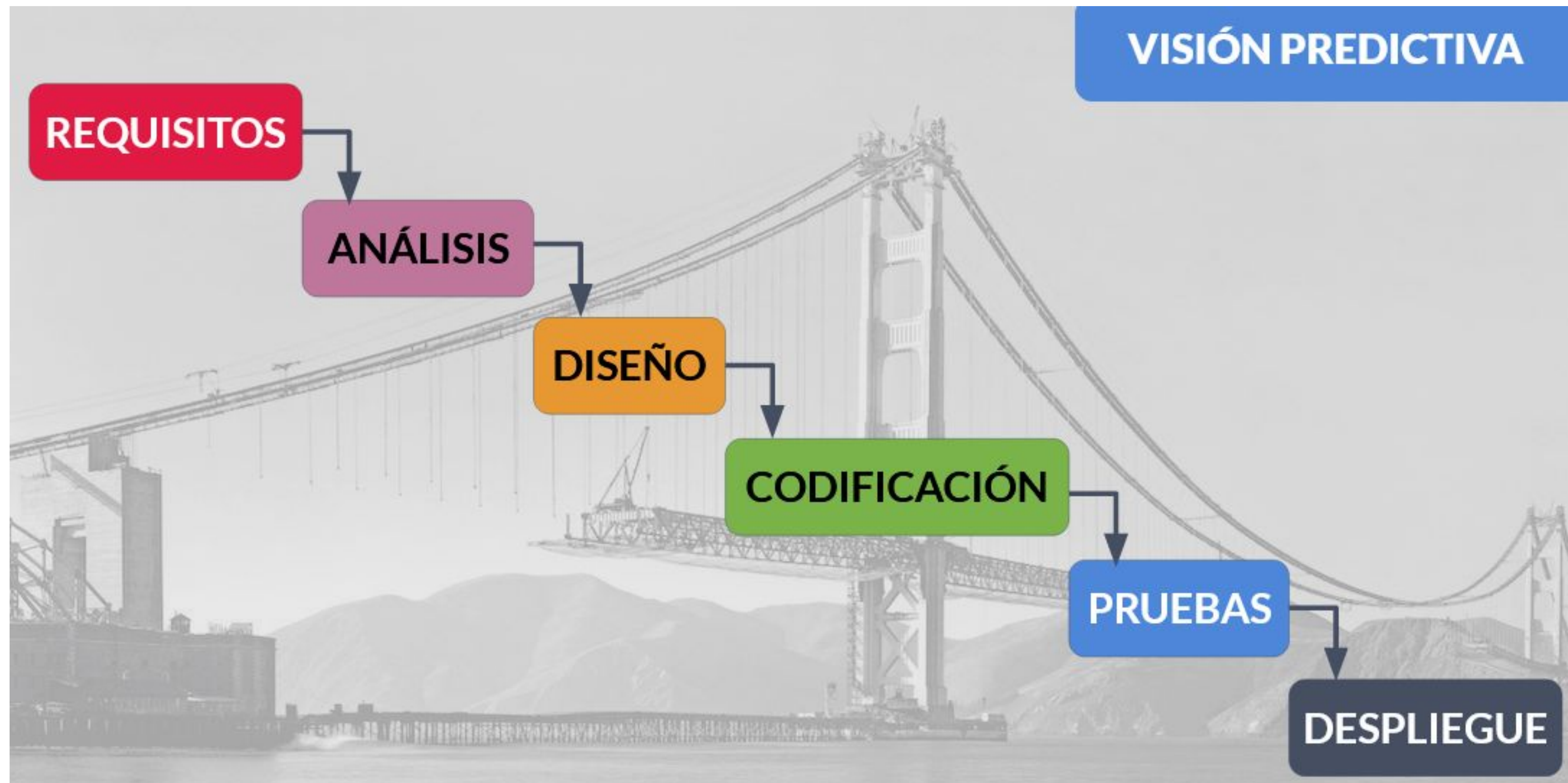
Fueron tres los ingredientes de la fórmula anti-crisis del software:

- ❑ Aplicar el conocimiento de **gestión de proyectos** que estaba cristalizando en esas mismas fechas
- ❑ Desarrollo de conocimiento y pautas de ingeniería para lograr garantías de **previsibilidad** y calidad a su desarrollo
- ❑ Producción basada en **procesos sistemáticos**

Aplicar el conocimiento de gestión de proyectos que estaba cristalizando en esas mismas fechas... y llegaron los **diagramas de Gantt** que desde 1915 eran ampliamente utilizando en la industria.

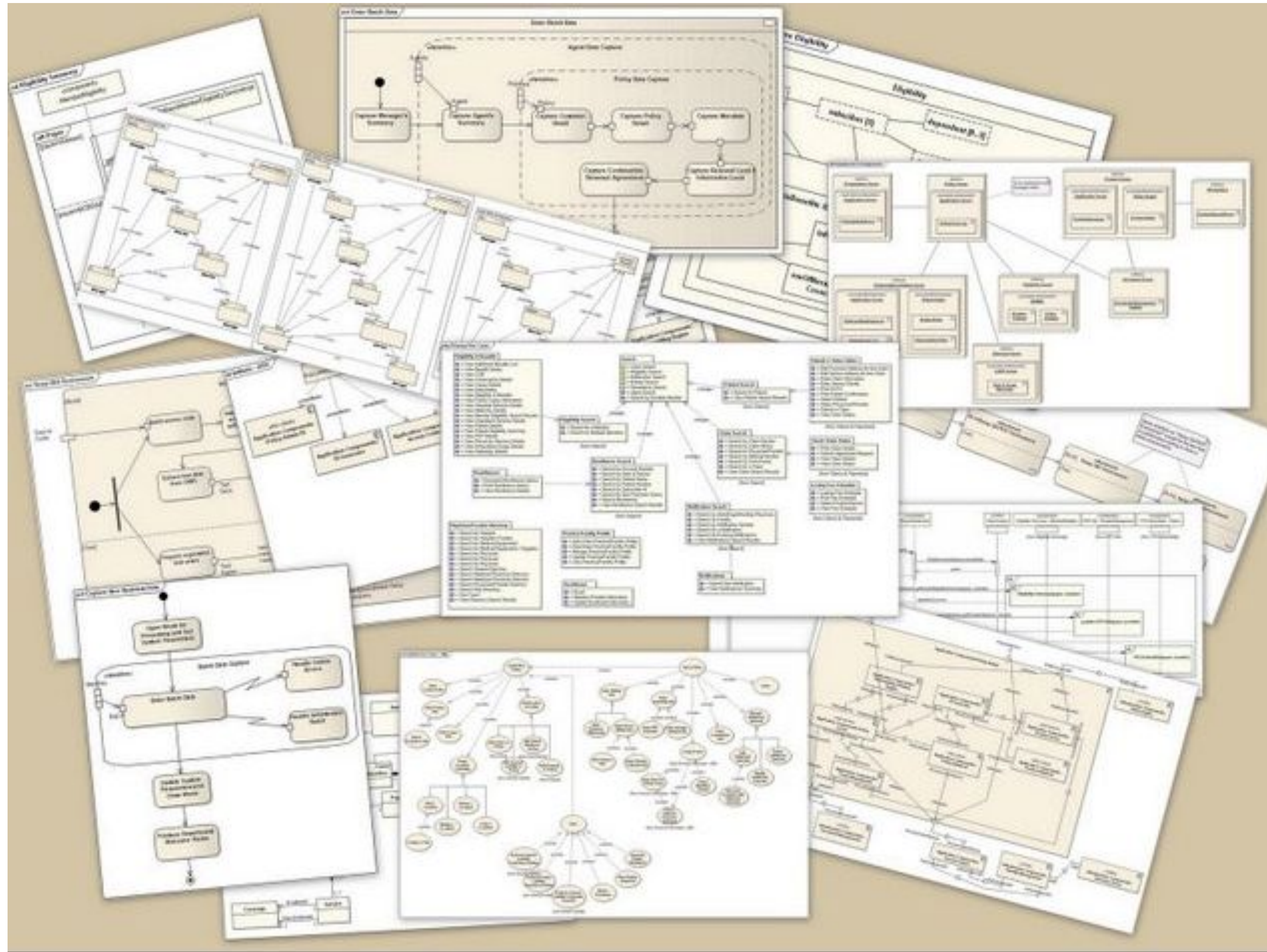


Desarrollo de conocimiento y pautas de ingeniería para lograr garantías de previsibilidad y calidad a su desarrollo... y llegó el **desarrollo “en cascada”**. Se pautó el desarrollo en fases. Cada fase no debe iniciarse sin haber terminado la anterior. Cada fase es desarrollada por un equipo de especialistas.

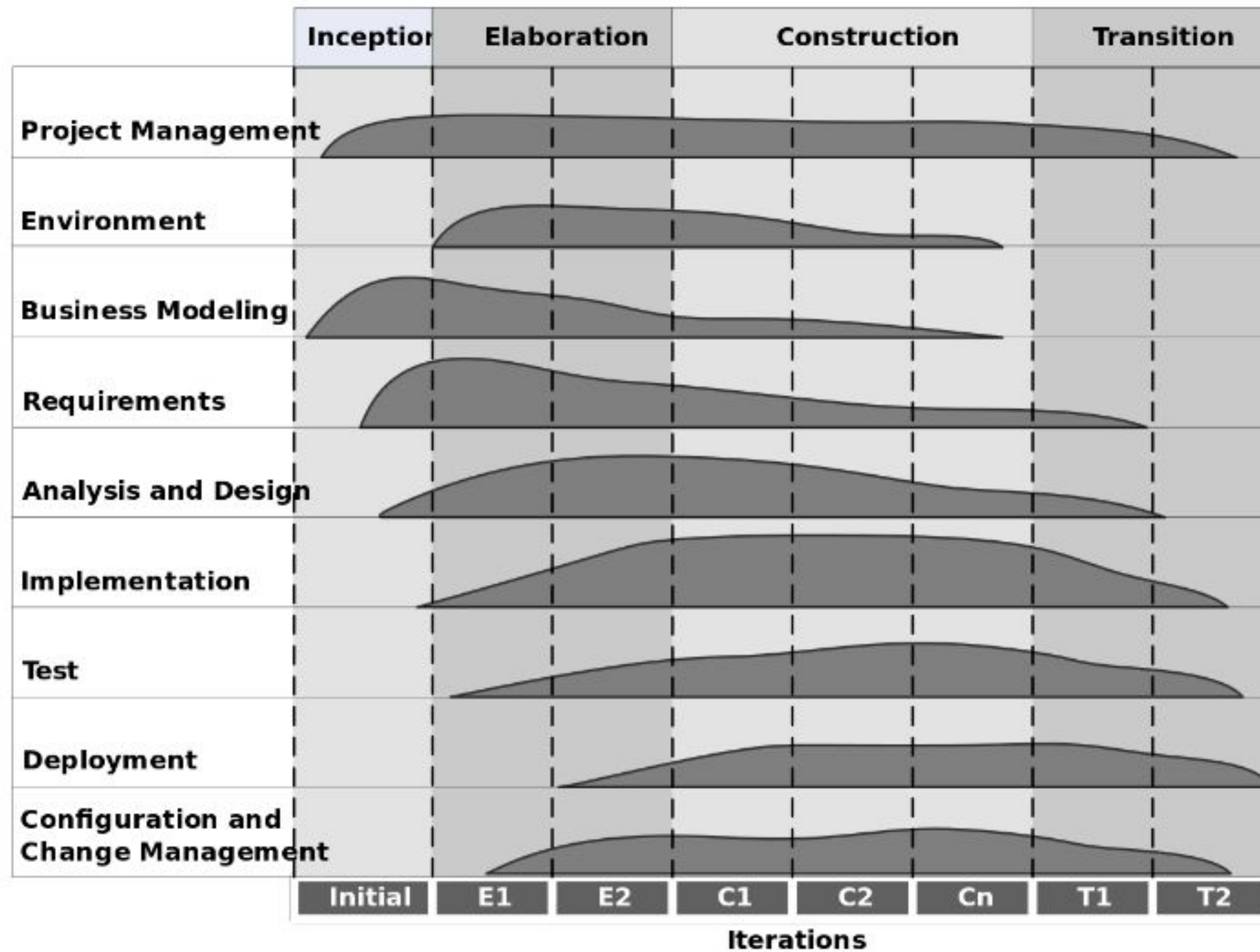


*Producción basada en procesos sistemáticos ... y se propusieron las “**Factorías de Software**”. Los arquitectos diseñan los “planos” del software y los programadores, entendidos como operarios, se limitan a codificarlos.*





UML. Lenguaje de modelado para dibujar “los planos” del software, imprescindibles para dar soporte al Proceso.



UP. Proceso Unificado.

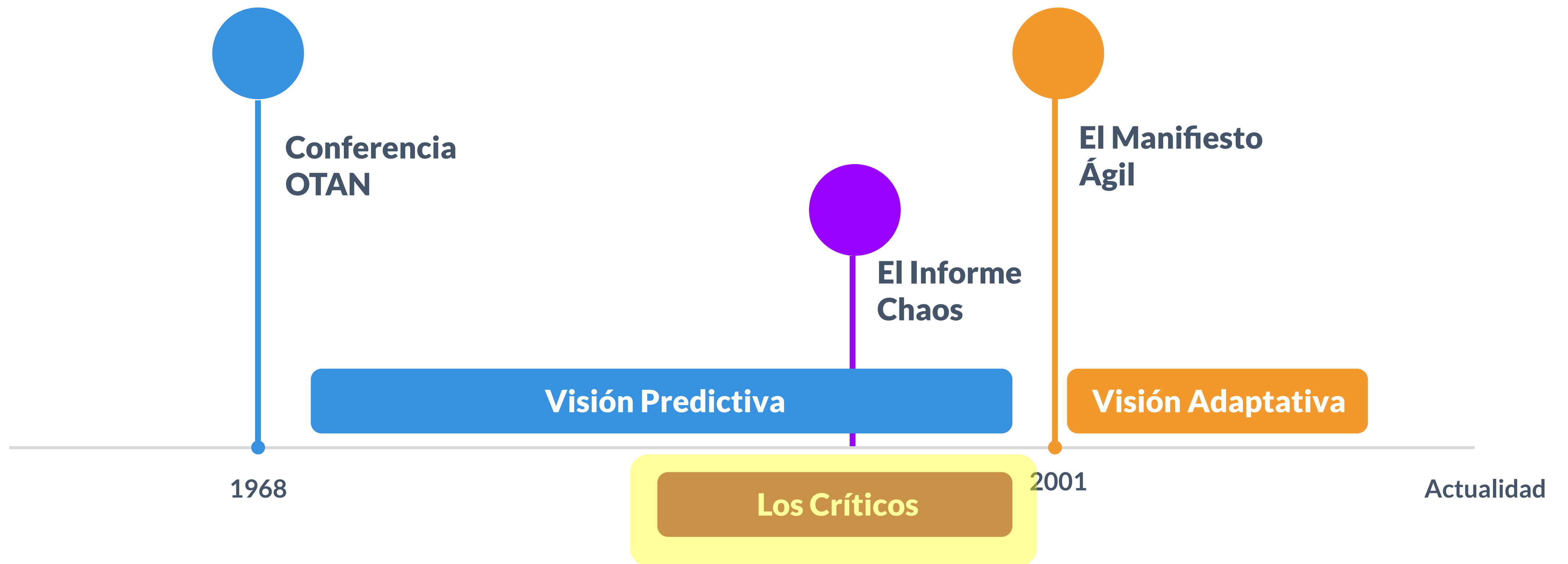
Es un marco de desarrollo de software que se caracteriza por estar dirigido por casos de uso, centrado en la arquitectura y por ser iterativo e incremental. El refinamiento más conocido y documentado del Proceso Unificado es el Proceso Unificado de Rational (RUP) o simplemente UP.

Área de Proceso	Categoría	Nivel de Madurez
Análisis Causal y Resolución (CAR)	Soporte	5
Gestión de Configuración (CM)	Soporte	2
Análisis de Decisiones y Resolución (DAR)	Soporte	3
Gestión Integrada del Proyecto (IPM)	Gestión de proyectos	3
Medición y Análisis (MA)	Soporte	2
Definición de Procesos de la Organización (OPD)	Gestión de procesos	3
Enfoque en Procesos de la Organización (OPF)	Gestión de procesos	3
Gestión del Rendimiento de la Organización (OPM)	Gestión de procesos	5
Rendimiento de Procesos de la Organización (OPP)	Gestión de procesos	4
Formación en la Organización (OT)	Gestión de procesos	3
Integración del Producto (PI)	Ingeniería	3
Monitorización y Control del Proyecto (PMC)	Gestión de proyectos	2
Planificación del Proyecto (PP)	Gestión de proyectos	2
Aseguramiento de la Calidad del Proceso y del Producto (PPQA)	Soporte	2
Gestión Cuantitativa del Proyecto (QPM)	Gestión de proyectos	4
Desarrollo de Requisitos (RD)	Ingeniería	3
Gestión de Requisitos (REQM)	Gestión de proyectos	2
Gestión de Riesgos (RSKM)	Gestión de proyectos	3
Gestión de Acuerdos con Proveedores (SAM)	Gestión de proyectos	2
Solución Técnica (TS)	Ingeniería	3
Validación (VAL)	Ingeniería	3
Verificación (VER)	Ingeniería	3

CMMI. CMMI pretende ser un modelo con las mejores prácticas para proveer a las organizaciones de aquellos elementos que son esenciales para que los procesos de negocio de las mismas sean efectivos.

CMMI evalúa el grado de cumplimiento de las mejores prácticas en el desarrollo de los procesos de la empresa.

Del Apollo, al Ejército de Simios



“Lo que un programador hace en un mes, dos programadores lo pueden hacer en dos meses”

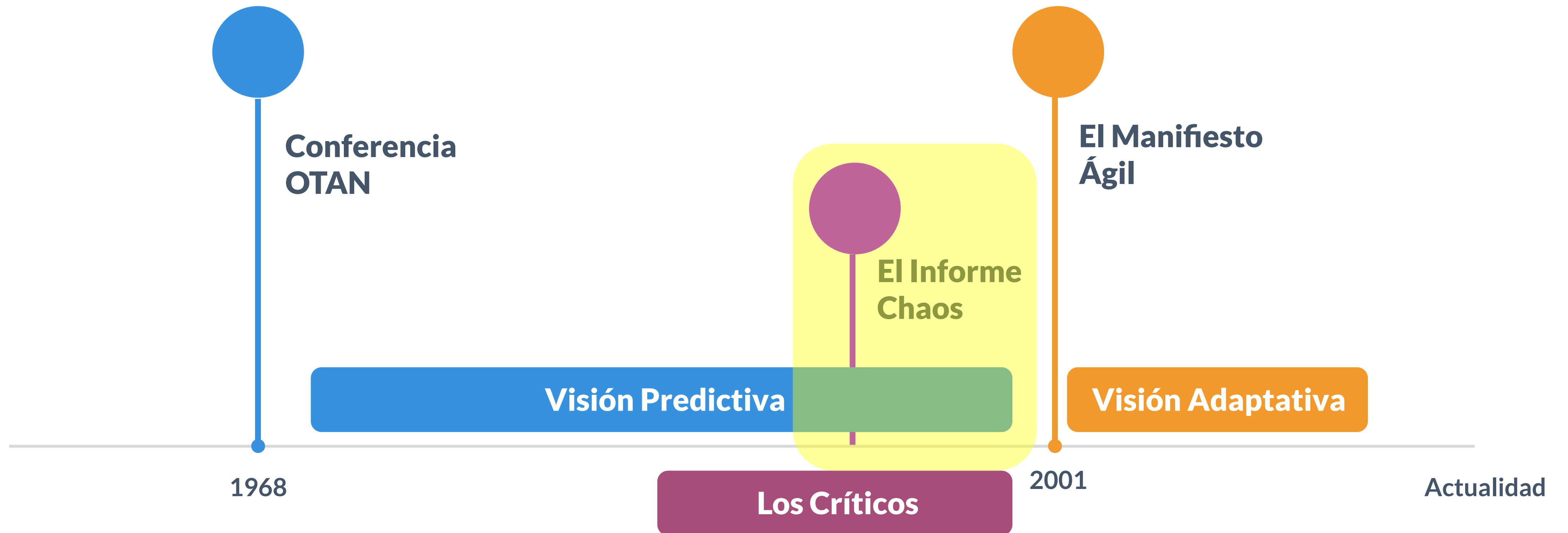
- Fred Brooks, *Ley de Brooks*



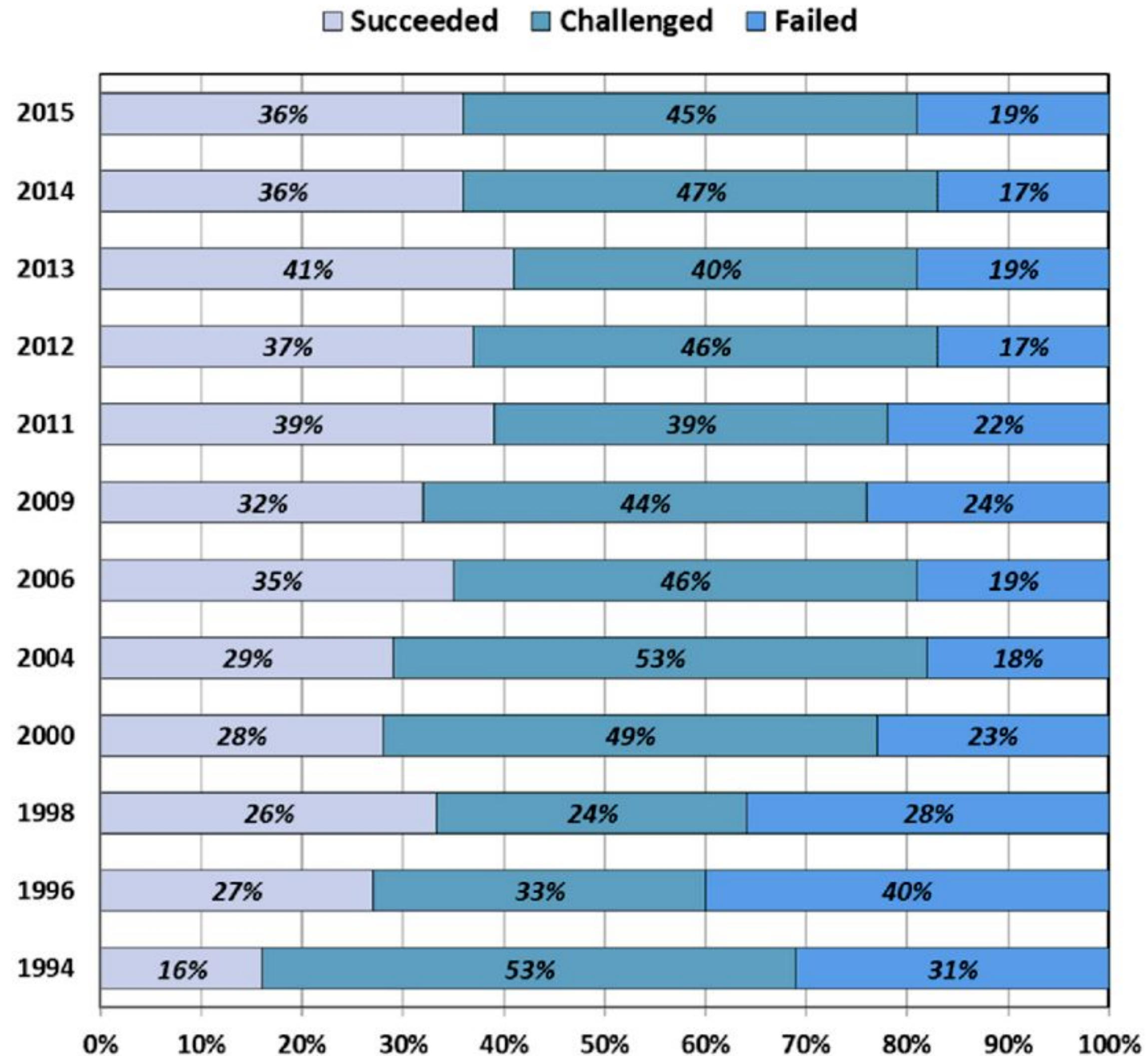
En su libro “**El Mítico Hombre-Mes**” (1975), Brooks introdujo algunos antipatrones de la industria como la “*bala de plata*” o la equivocada creencia de que al aumentar el número de personas trabajando en un proyecto se logrará acortar los plazos. También introduce conceptos fundamentales como la complejidad “*esencial*” y la “*accidental*” del código.

Resulta impensable que un editor proponga a un escritor añadir más escritores al “equipo” con el objetivo de acortar los plazos de publicación de su nueva **novela**. El esfuerzo que supone alinear a los escritores en el estilo narrativo o la trama de la novela, hace inviable este tipo de soluciones. Cuando el producto requiere de esfuerzo intelectual o creativo añadir más miembros al equipo no tiene por qué acortar los plazos.

Del Apollo, al Ejército de Simios



El Informe CHAOS



Resultados del Informe CHAOS desde 1994

Debido a la alta percepción de fracaso en los proyectos software, la consultora especializada en gestión de proyectos software “Standish Group” realizó una investigación cuantitativa sobre el éxito de estos proyectos en el año **1994**, el resultado fue el **Informe CHAOS**. Este informe cosechó gran repercusión, poniendo de manifiesto la necesidad de implantar nuevas formas de gestionar los proyectos software.

A finales de los 90, reputados profesionales con predicamento en diferentes foros técnicos, comenzaron a cuestionar las metodologías formales, que representadas por CMM e ISO, y respaldadas por la autoridad y los medios de sus respectivas organizaciones, estaban asentando una ingeniería del software basada en procesos.

Madrid ‘entierra’ 10 millones en un programa informático que no funciona



Era el sistema informático llamado a unificar toda la gestión tributaria que se tramita en el Ayuntamiento de Madrid. Sin embargo, después de seis años de desarrollo, dos contratos con empresas tecnológicas y diez millones de euros de inversión, el programa que en 2006 anunció a bombo y platillo el entonces alcalde de Madrid, Alberto Ruiz Gallardón ha quedado en agua de borrajas.

Hertz pierde la paciencia con Accenture: demanda de 40 millones de dólares por una web que no funciona

Publicado el 24 abril, 2019 por Rodolfo de Juana

Knight Capital, 2012

El fallo de software que propició un desastre bursátil de 440 millones de dólares en 45 minutos

- ▶ Un fallo hizo que millones de operaciones de compraventa se volvieran locas
- ▶ Las acciones de más de 140 compañías cotizaron a precios inusuales
- ▶ Cuando la empresa de *trading* Knight Capital arregló el fallo ya había perdido 440 millones

healthcare.gov, 2013

■ 18 de octubre de 2013 0:51 CEST

The Obamacare Website Didn't Have to Fail. How to Do Better Next Time

- The flawed software that doomed healthcare.gov and how to fix it

Comunidad de Madrid, 2010

La Comunidad de Madrid pagó 5,5 millones a Indra por un 'software' que nunca entregó

- ⤵ Adjudicó en 2010 el desarrollo de un programa de gestión educativa por 15,8 millones y, tras abonar un tercio, resolvió el contrato después de que la empresa adujera problemas técnicos irresolubles

Scrum, 1993.

Su creadores fueron Jeff Sutherland y Ken Schwaber. Analizando estudios en los que se indagaban las formas de trabajo de equipos de éxito, destilaron sus reglas para ponerlas al servicio de la Ingeniería del Software.

Scrum prescribe únicamente tres roles (Scrum Master, Product Owner y Equipo de Desarrollo), apenas dos liturgias (Dailys y Retrospectivas) y tres entregables (el Backlog, los diagramas de Burndown y, por supuesto, el Software Funcionando).

“Cualquier SCRUM sin un producto al final del sprint, es un SCRUM fallido.”

- Jeff Sutherland



“No soy un gran programador, soy un buen programador con grandes hábitos.”

- Kent Beck



Extreme Programming (XP), 1999.

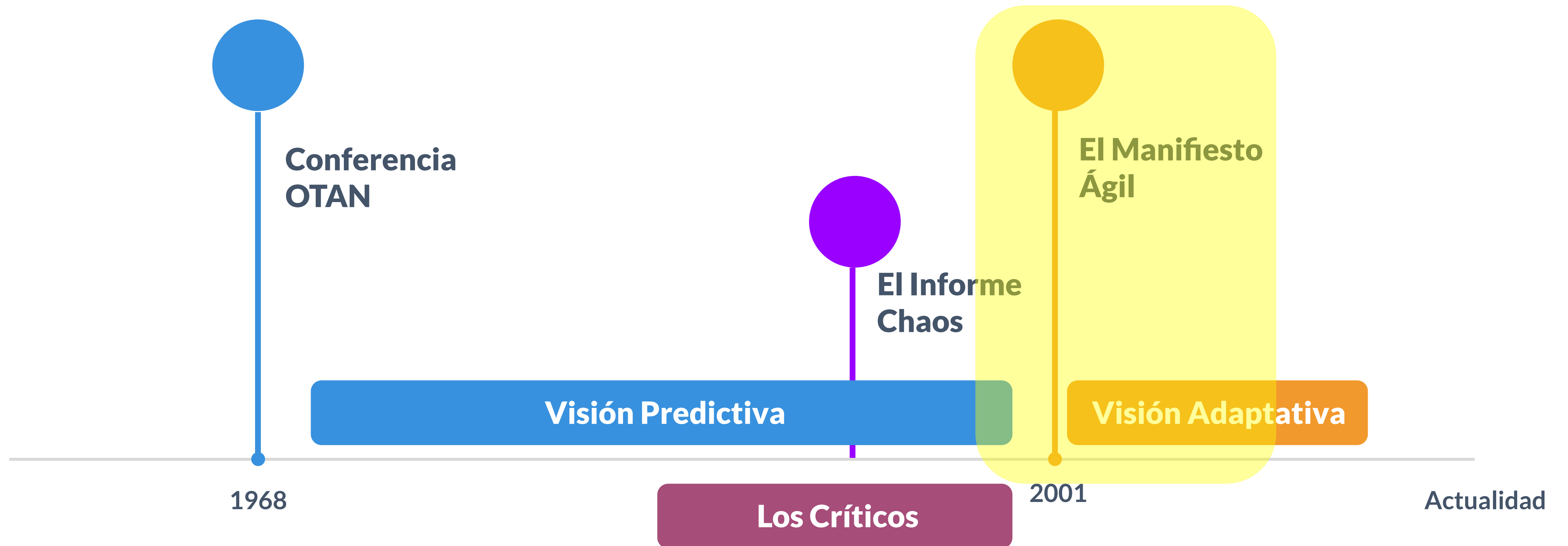
Su creador fue Kent Beck. Es la metodología ágil más popular, y posiblemente también la más transgresora de la ortodoxia basada en procesos. Extreme Programming (XP) se basa en la suposición de que es posible obtener software de gran calidad a pesar, o incluso como consecuencia del cambio continuo. El desarrollo guiado por pruebas (**TDD**), que prescribe XP, fue toda una revolución. El diseño del software emerge de las pruebas, no de los “planos”.



La industria manufacturera ha conseguido sistematizar sus metodologías de construcción dotando de previsibilidad al proceso y generando productos de calidad y económicamente rentables, ¿debería la recién definida Ingeniería de Software asimilar dichas metodologías para construir productos software de calidad y baratos? ¿debemos construir software de la misma forma que construimos puentes?

- A) **Sí.** El producto software debe construirse igual que se construyen coches, casas o puentes.
- B) **No.** Debemos idear una nueva metodología de construcción, una nueva forma de desarrollo de software que nos permita crear productos de calidad, ajustados a presupuestos y en plazos.

Del Apollo, al Ejército de Simios



El Manifiesto Ágil



En marzo de 2001, 17 críticos de estos modelos, convocados por Kent Beck, que acababa de definir una nueva metodología denominada Extreme Programming, se reunieron en Salt Lake City para discutir sobre los modelos de desarrollo de software. El manifiesto ágil surgió con espíritu desafiante y beligerante contra los modelos predictivos.

El Manifiesto Ágil escrito por primera vez

El Manifiesto Ágil

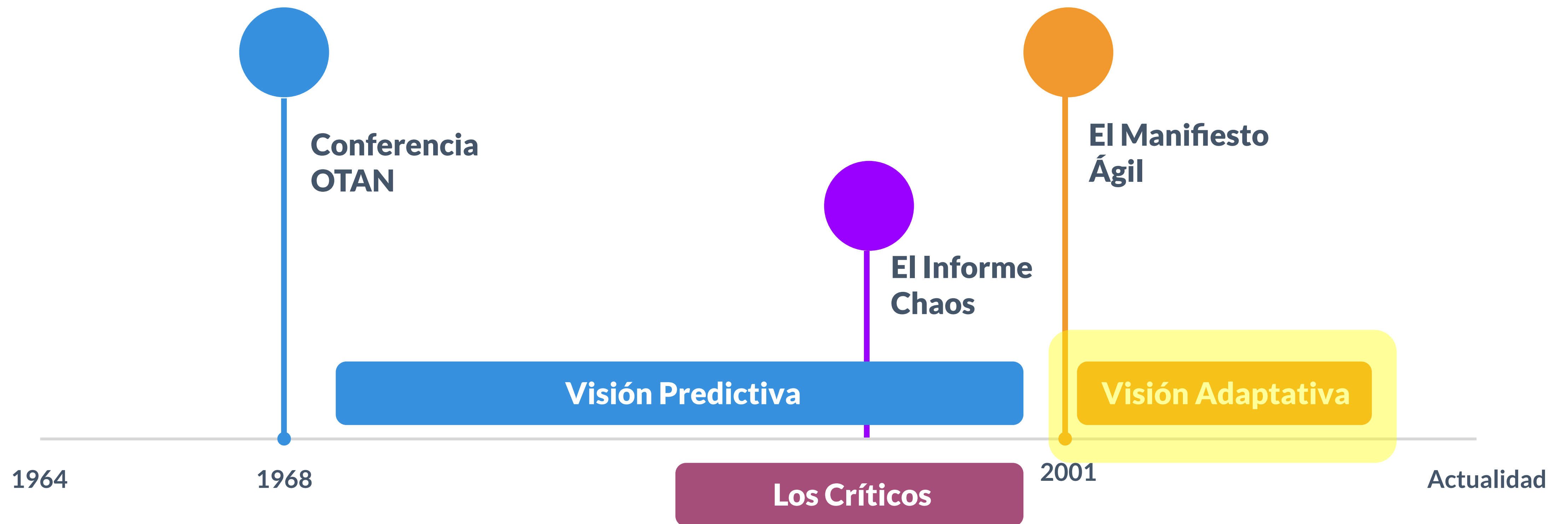
“Estamos poniendo al descubierto mejores métodos para desarrollar software, haciéndolo y ayudando a otros a que lo hagan. Con este trabajo hemos llegado a valorar:

- *A los **individuos y su interacción**, por encima de los procesos y las herramientas.*
- *El **software que funciona**, por encima de la documentación exhaustiva.*
- *La **colaboración con el cliente**, por encima de la negociación contractual.*
- *La **respuesta al cambio**, por encima del seguimiento de un plan.*

Aunque hay valor en los elementos de la derecha, valoramos más los de la izquierda.”

- Kent Beck, Mike Beedle, Arie van Bennekum, Alistair Cockburn, Ward Cunningham, Martin Fowler, James Grenning, Jim Highsmith, Andrew Hunt, Ron Jeffries, Jon Kern, Brian Marick, Robert C. Martin, Steve Mellor, Ken Schwaber, Jeff Sutherland, Dave Thomas.

Del Apollo, al Ejército de Simios



Not like this....



1



2



3



4

Like this!



1



2



3



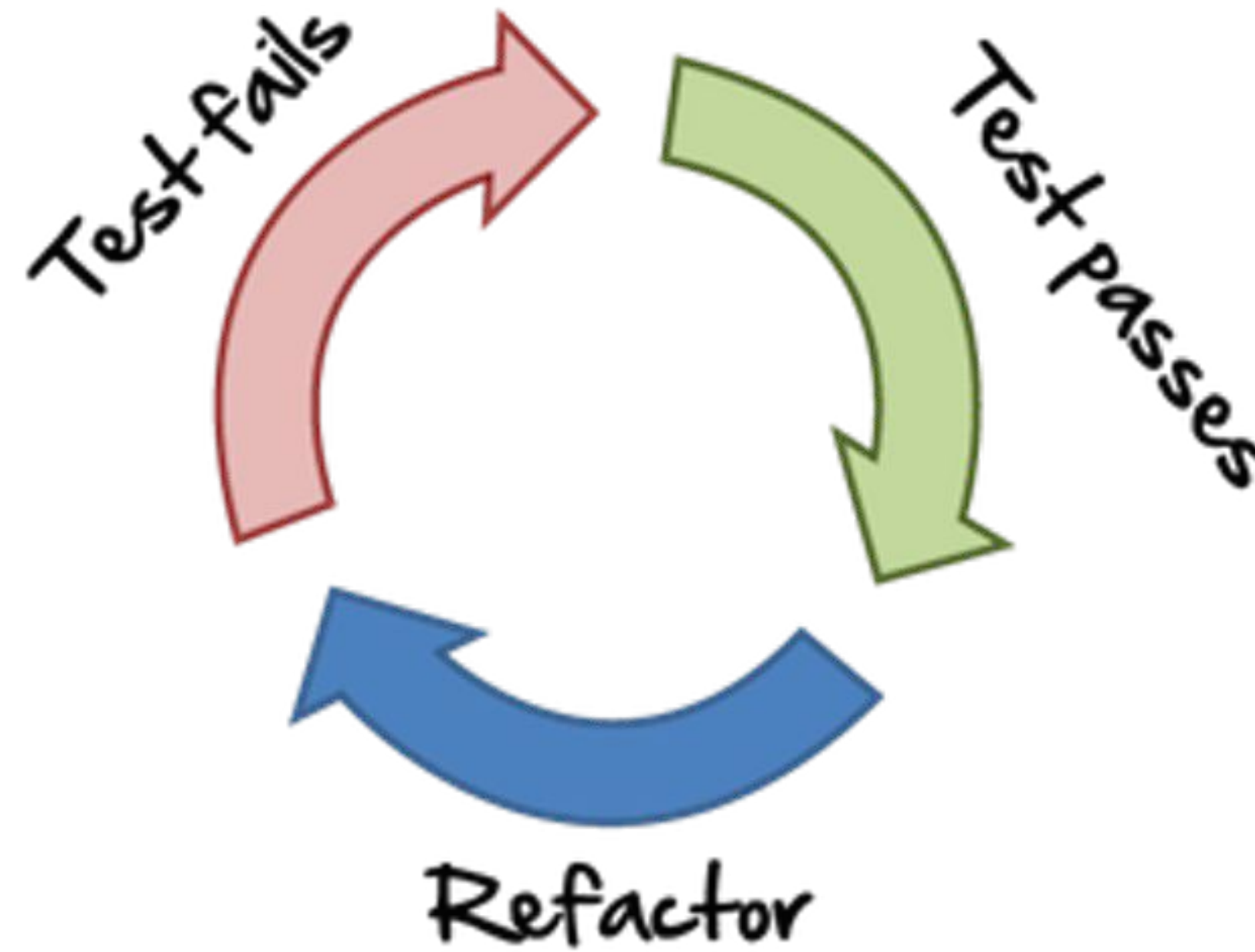
4



5

Las metodologías ágiles persiguen la entrega iterativa y temprana de producto con valor funcional.

El **desarrollo guiado por pruebas** permite prescindir de los planos del código facilitando de esta forma el cambio.



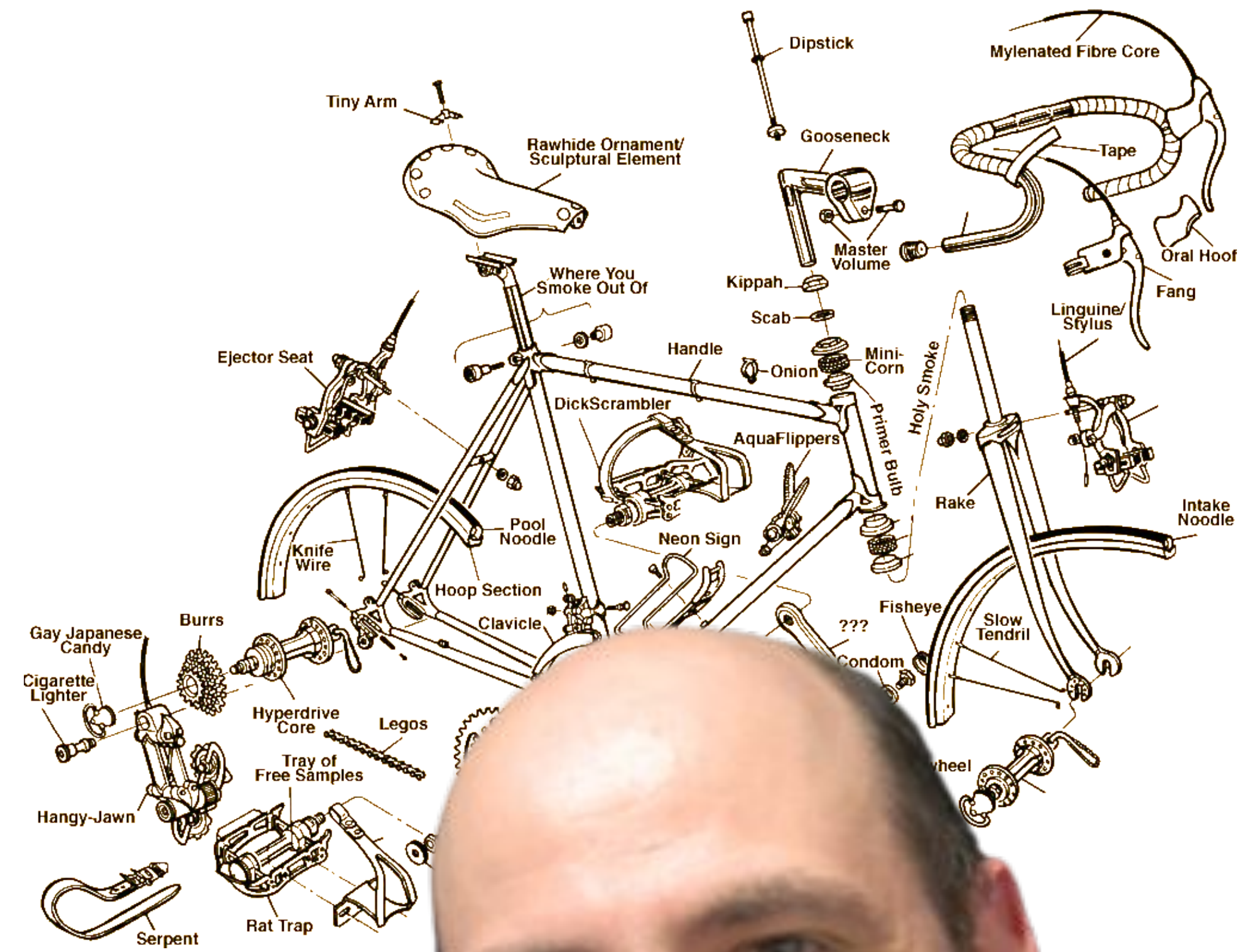


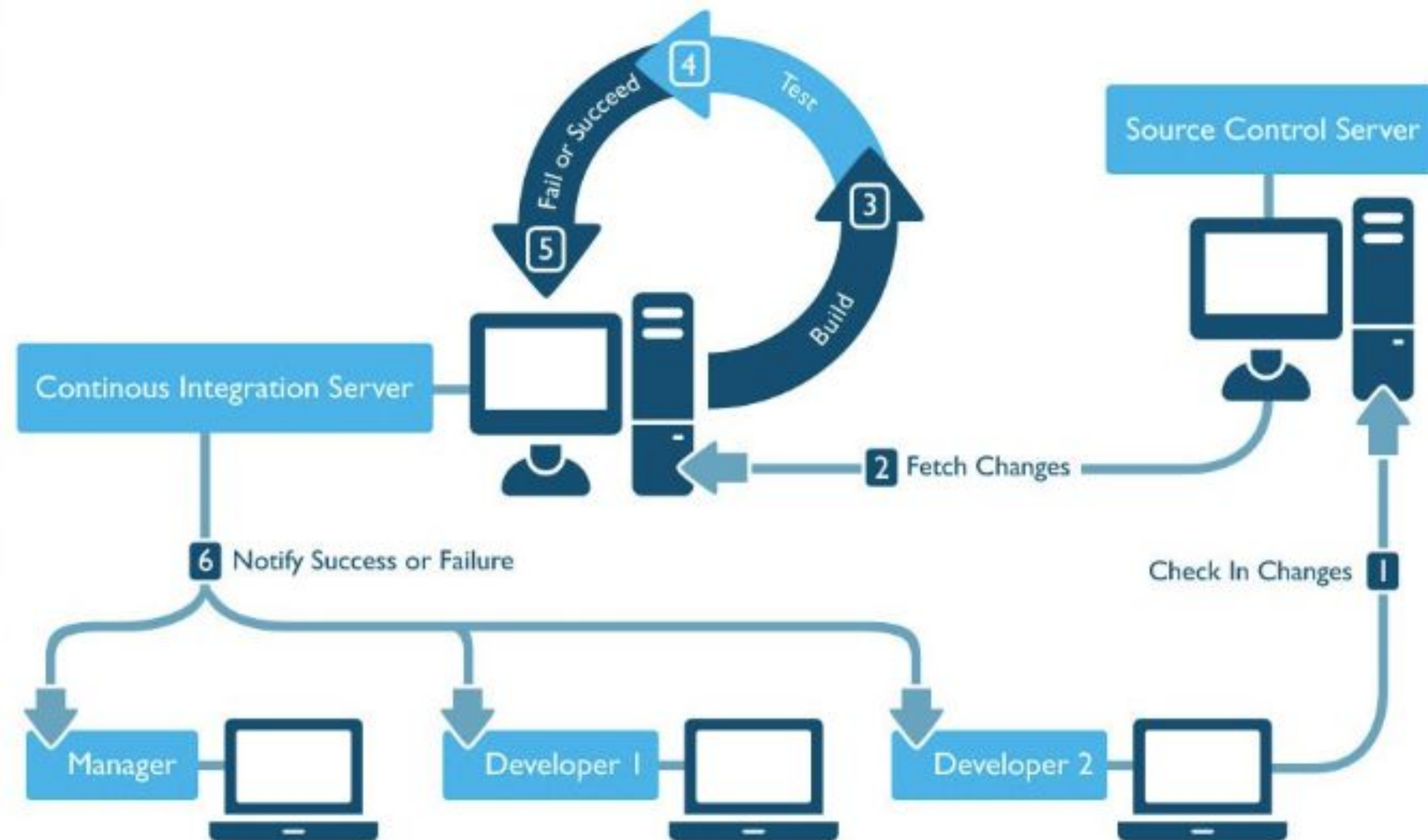
El poder de la automatización

La industria ha adoptado nuevas técnicas y prácticas para la construcción del software muy alineadas con la visión adaptativa. Estas prácticas permiten la **automatización** de muchos procesos. La automatización ha llegado a la ingeniería del software pero no en la dirección prevista por los primeros ingenieros de los 70s.

Contenedores de dependencias, 2004.

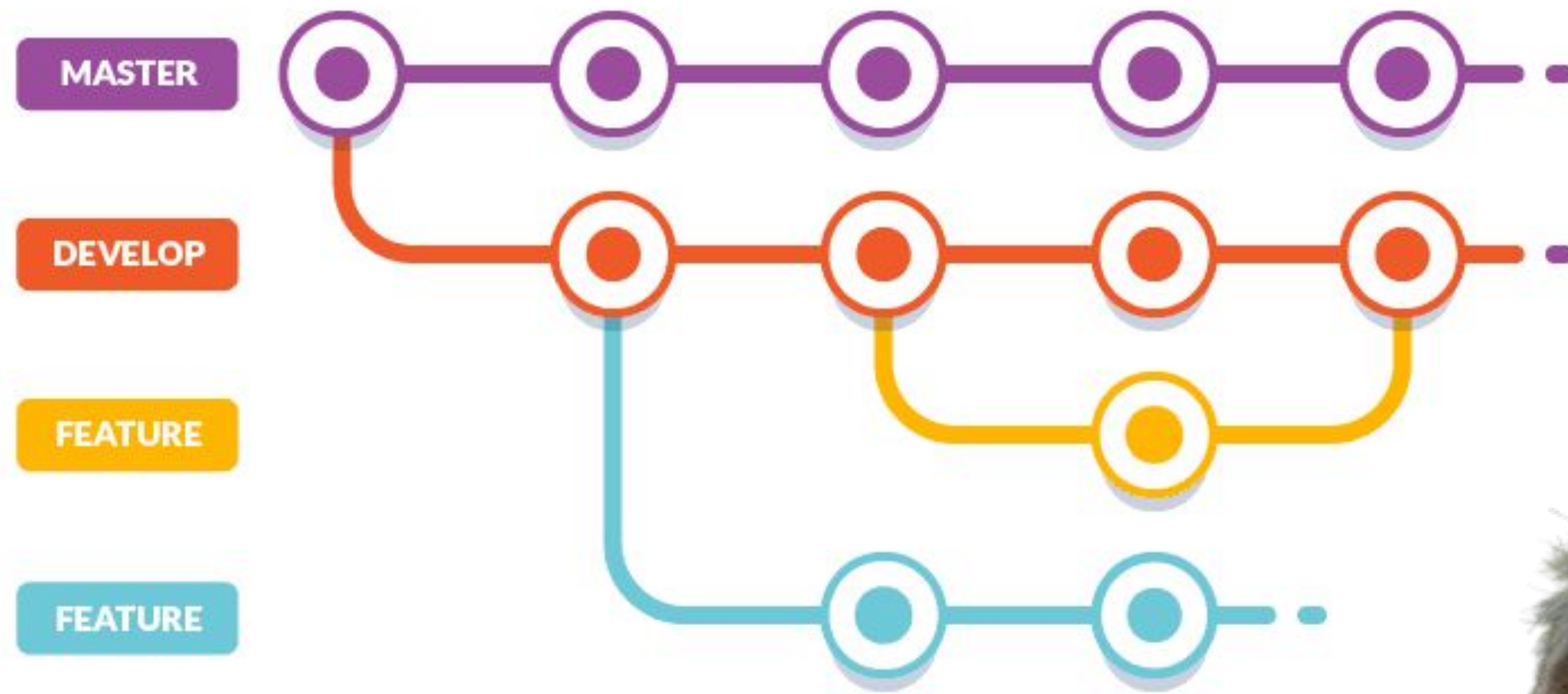
Propuestos por Martin Fowler en el 2004. Haciendo uso de la enorme potencia que el polimorfismo y la abstracción brinda a los diseños de software, es posible registrar en un “contenedor de dependencias” cada una de las “piezas” (clases) que componen el software. Si cada pieza indica las dependencias que tiene con otras piezas, el contenedor será capaz de ensamblar el producto. De esta forma el desarrollador podrá dotar al producto de novedosos comportamientos únicamente modificando el registro de piezas, sustituyendo unas piezas por otras.





Integración Continua, 2000.

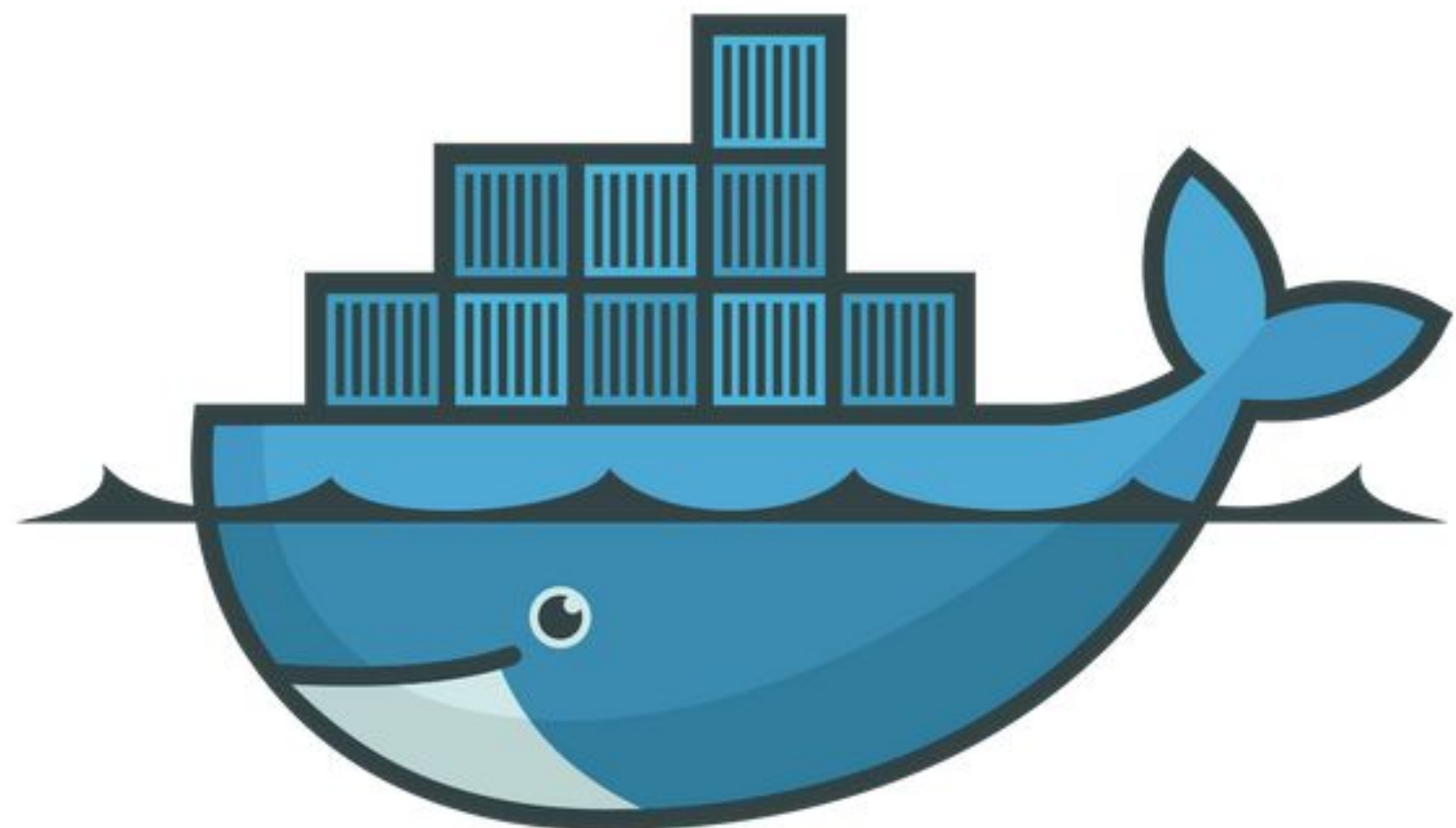
Propuesta por Martin Fowler en el 2000. La plataforma de integración periódicamente o a petición de un miembro del equipo de desarrollo descarga el código fuente del sistema de control de versiones, lo compila, lanza las pruebas automatizadas, mide la calidad del código y despliega los ejecutables.



Git (2007) y Git WorkFlow (2011).

Establecer un flujo normalizado de trabajo en relación al sistema de control de versiones ayuda al equipo de desarrollo a trabajar en diferentes versiones del mismo producto de forma simultánea.





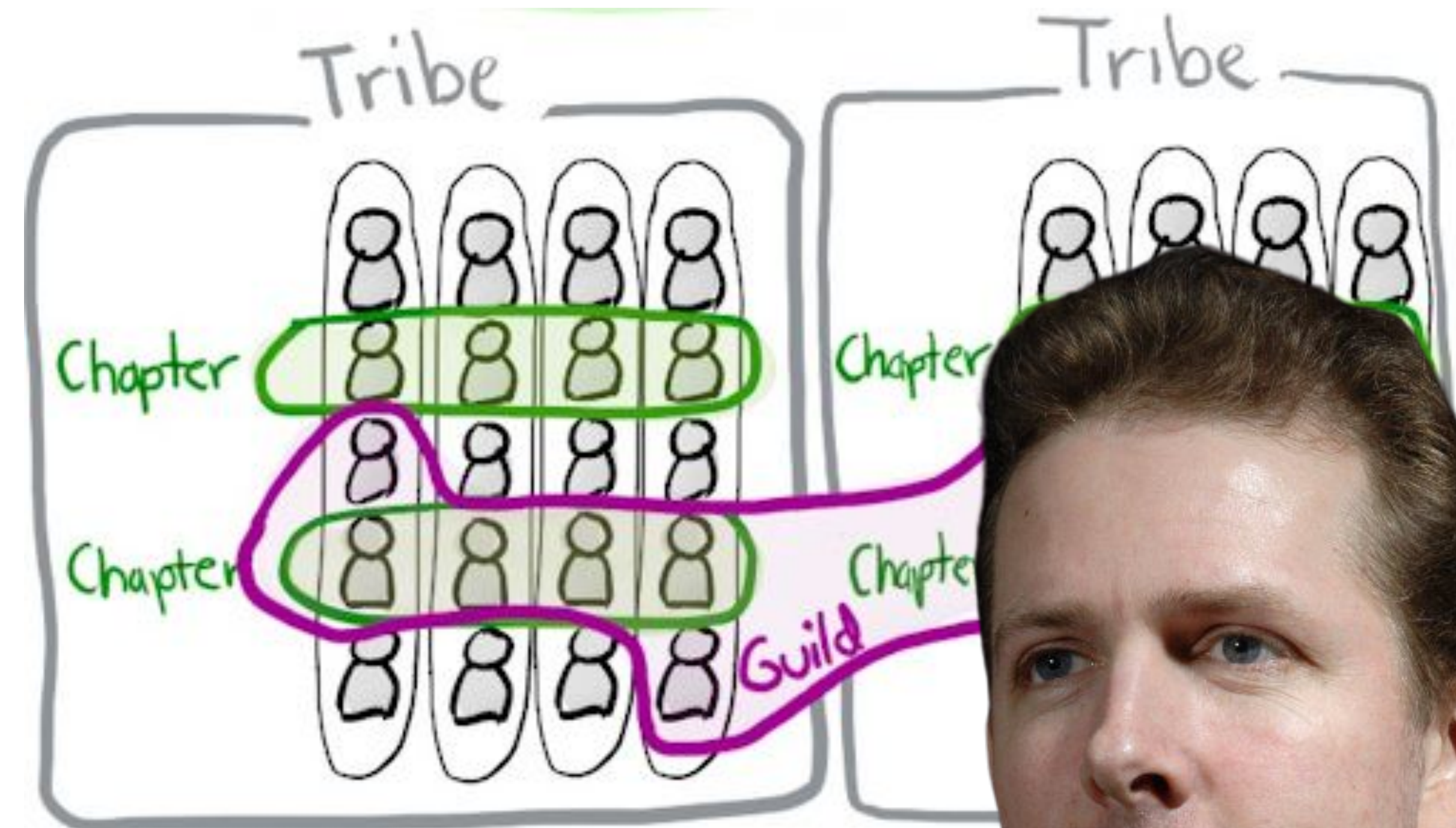
Contenedores de aplicaciones, 2013.

Permiten automatizar el despliegue de aplicaciones dentro de contenedores de software, proporcionando una capa adicional de abstracción y automatización. Al virtualizar la aplicación es posible su despliegue en múltiples sistemas operativos y servidores.

Ingeniería del Software **en la actualidad**

La industria está cada vez más entregada a la búsqueda de la excelencia en el código, a las buenas prácticas, al testing automatizado y a la entrega continua y temprana de producto con valor.

Henrik Kniberg ideó una estrategia para **escalar Scrum** en grandes organizaciones denominada “Modelo Spotify”.



Ingeniería del Software **en la actualidad**

NETFLIX

El “Ejército de Simios” de Netflix es una buena prueba del esfuerzo que las grandes corporaciones ponen en acortar sus tiempos de respuesta automatizando sus procesos.

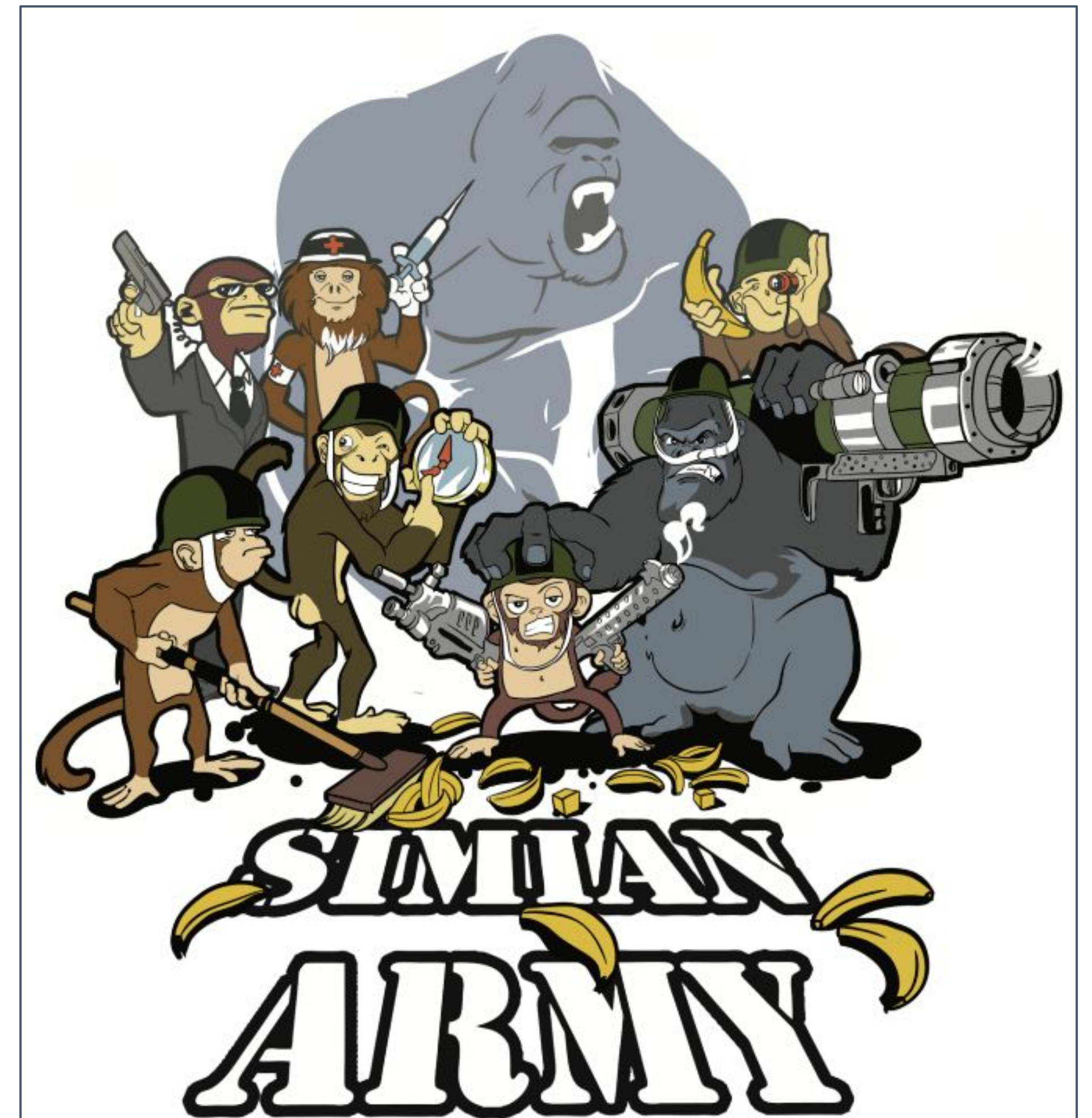


TABLA DE CONTENIDOS

- 01** Breve Historia
- 02** ¿Qué es la Ingeniería del Software?
- 03** Ciclo de vida
- 04** Herramientas CASE
- 05** Diagramas

¿Qué es la Ingeniería del Software?

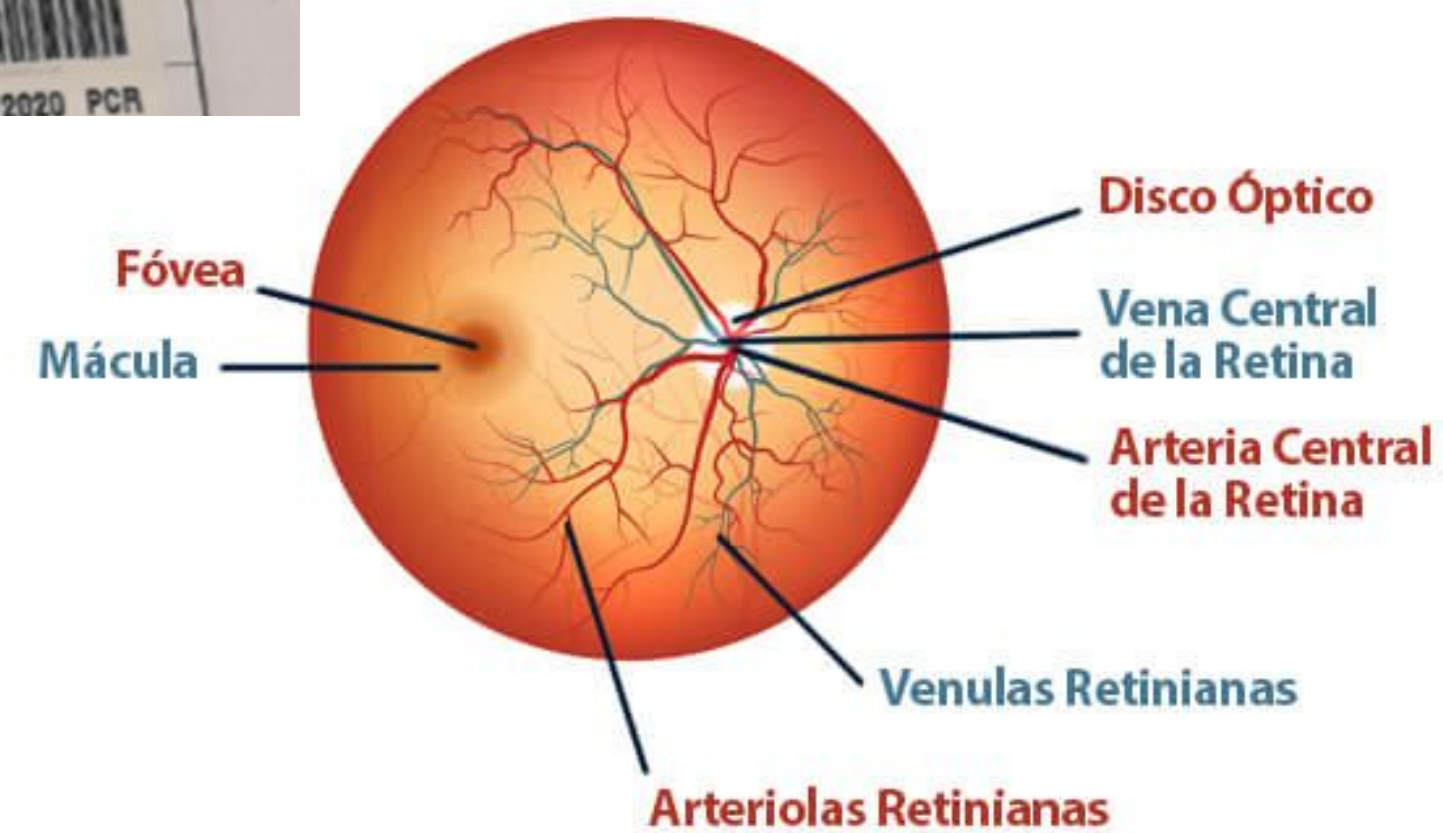
Software

Pool SARS-CoV2
 Fecha: 27/07/2020 - 30/7/2020. Extractor + Reactivo: STARLET COVID-19/2019-NCov-2

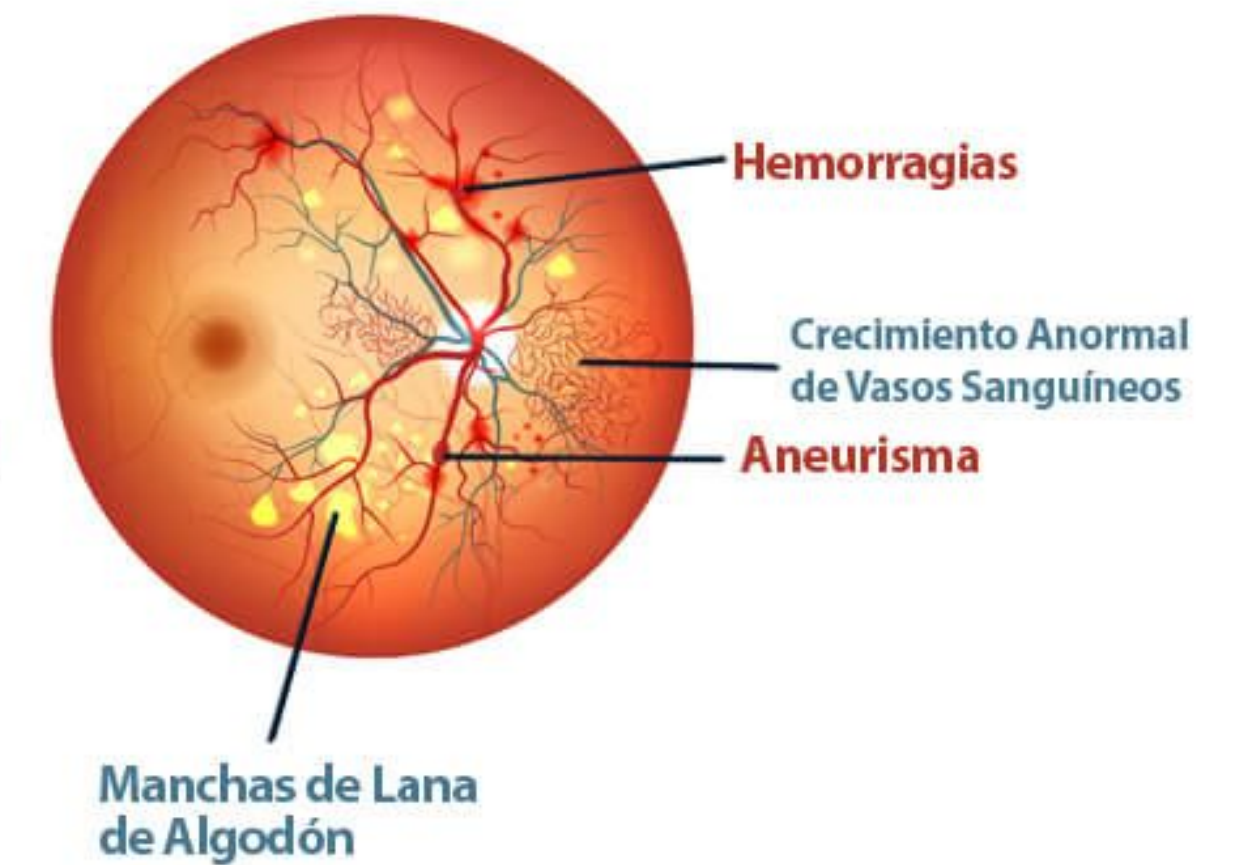
SEPARADAS

Pool	Muestra 1	Muestra 2	Muestra 3	Muestra 4	Muestra 5
P <u>1</u>	30/07/2020 PCR 35029521-1	30/07/2020 PCR 35029529-1	30/07/2020 PCR 35029506-1	30/07/2020 PCR 35028986-1	30/07/2020 PCR 35028976-1
P <u>2</u>	30/07/2020 PCR 76683439-1	30/07/2020 PCR 35029530-1	30/07/2020 PCR 35029505-1	30/07/2020 PCR 35028985-1	30/07/2020 PCR 35028975-1
P <u>3</u>	30/07/2020 PCR 76683438-1	30/07/2020 PCR 35029527-1	30/07/2020 PCR 35029504-1	30/07/2020 PCR 35028984-1	30/07/2020 PCR 35028973-1
P <u>4</u>	30/07/2020 PCR 76683437-1	30/07/2020 PCR 35029531-1	30/07/2020 PCR 35028992-1	30/07/2020 PCR 35028983-1	30/07/2020 PCR 35028974-1
P <u>5</u>	30/07/2020 PCR 35029523-1	30/07/2020 PCR 35029526-1	30/07/2020 PCR 35028991-1	30/07/2020 PCR 35028982-1	30/07/2020 PCR 35028972-1
P <u>6</u>	30/07/2020 PCR 35029522-1	30/07/2020 PCR 35029533-1	30/07/2020 PCR 35028990-1	30/07/2020 PCR 35028981-1	30/07/2020 PCR 35028971-1

Ojo Sano



Retinopatía Diabética





Ingeniería

Conjunto de conocimientos y técnicas científicas aplicadas a la creación, perfeccionamiento e implementación de estructuras (tanto físicas como teóricas) para la resolución de problemas que afectan a la actividad cotidiana de la sociedad.



¿Qué es la ingeniería al software?

Artesania



VS

Ingenieria





¿Qué es la ingeniería al software?

“La programación es el acto inmediato de producir código. La ingeniería de software es el conjunto de políticas, prácticas y herramientas que son necesarias para hacer que ese código sea útil durante el tiempo que sea necesario y que permita la colaboración entre el equipo”

*“[...] La programación es ciertamente una parte fundamental de la ingeniería del software. [...] necesitamos distinguir entre las tareas de programación (desarrollo) y las tareas de ingeniería de software (desarrollo, modificación, mantenimiento). El tiempo añade una importante nueva dimensión a la programación. **Los cubos no son cuadrados, la distancia no es velocidad. La ingeniería de software no es programación**”*

- Software Engineering at Google (2020)



¿Qué es la ingeniería al software?

Construir software es mucho más que únicamente programar. Una buena prueba de ello es la cantidad de roles que intervienen en la construcción de un producto software.

Roles técnicos que definen cómo se construye el software:

- Junior/Mid/Senior Front/Back/Fullstack Developer
- Team Lead
- Engineering Manager
- Architect/Principal
- CTO

Roles de negocio que definen qué software se construye:

- Analyst/ Head Analyst
- UX/UI designer
- Product/Project Manager
- CPO



¿Qué es la ingeniería al software?

